ISEP INSTITUTO SUPERIOR DE ENGENHARIA DO PORTO

Introduction to PHP

Sistemas de Informação 1

Credits and Disclaimer

- This material/slides are adapted from:
  - SINF1 2022/23 slides produced by Constantino Martins
  - ARQSI 2014/15 slides
  - Books
  - Web sites:
    - https://www.php.net/;
    - https://www.w3schools.com/
    - ...

What is PHP?

- **PHP** is a recursive acronym for **PHP: Hypertext Preprocessor**
- It is a *widely-used open-source general-purpose* scripting language that is especially suited for web development and can be embedded into HTML
- PHP is a server-side scripting language
  - PHP scripts are executed on the server
- PHP supports many databases
  - MySQL, Oracle, PostgreSQL, …
- PHP is free to download and use

Is PHP dead?

1995: PHP is dead, learn ColdFusion
2002: PHP is dead, learn ASP.net
2003: PHP is dead, learn Django
2004: PHP is dead, learn Ruby on Rails
2010: PHP is dead, learn Flask
2011: PHP is dead, learn AngularJS
2016: PHP is dead, learn Next.js
2022: PHP is dead, learn Python
2023:

PHP Introduction

- PHP pages **mix HTML tags with script code** that controls the visual output of the page.
- The PHP code is enclosed in special start and special end processing instructions **<?php** and **?>** that allow to jump into and out of "*PHP mode*".

```
<html>
   <head><title>Example</title></head>
   <body>
       <?php
           echo "Hi.<br/><b>This is a PHP script!</b>";
       ?>
   </body>
</html>
```

- The <?php and ?> are called **PHP delimiters**.

PHP delimiters

- PHP delimiters are nothing but open-close tags to enclose PHP scripts. PHP code can be parsed if and only if it is enclosed with these delimiters.
- There are various sets of delimiters to recognize PHP code. These are as listed below.
  - **<?php** … **?>** – These are preferred as they explicitly identify the embedded code.
  - **<script language="php">** … **</script>** – like the previous ones.
  - **<?** … **?>** – Called PHP short tags which will work based on the value set with the *short_open_tag* directive of the PHP configuration file.
  - **<%** … **%>** – These are the delimiters for Active Server Pages(ASP) code. But it will be used for PHP code if and only if the *asp_tags* directive is enabled.

View PHP Settings

- phpinfo()
  - Outputs a large amount of information about the current state of PHP. This includes:
    - information about PHP compilation options and extensions,
    - the PHP version,
    - server information and environment (if compiled as a module),
    - the PHP environment,
    - OS version information,
    - paths,
    - master and local values of configuration options,
    - HTTP headers,
    - the GNU Public License

View PHP Settings

```
<html>
   <head><title>Example</title></head>
   <body>
      <?php
         phpinfo();
      ?>
   </body>
</html>
```

PHP Version 5.5.9-1ubuntu4.29

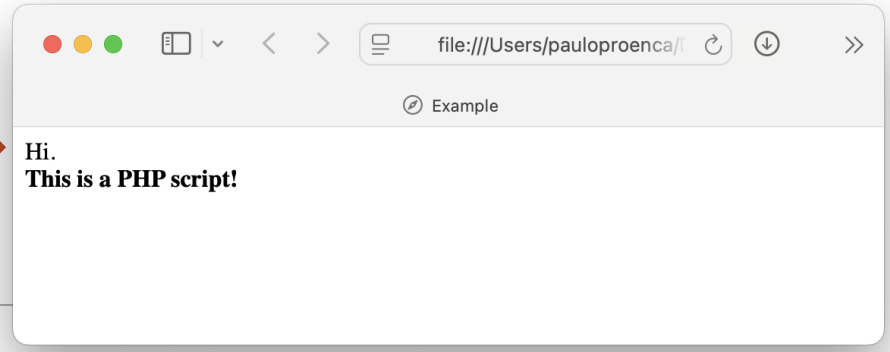
System	Linux www 3.13.0-170-generic #220-Ubuntu SMP Thu May 9 12:40:49 UTC 2019 x86_64
Build Date	Apr 22 2019 18:33:42
Server API	Apache 2.0 Handler
Virtual Directory Support	disabled
Configuration File (php.ini) Path	/etc/php5/apache2
Loaded Configuration File	/etc/php5/apache2/php.ini
Scan this dir for additional .ini files	/etc/php5/apache2/conf.d
Additional .ini files parsed	/etc/php5/apache2/conf.d/05-opcache.ini, /etc/php5/apache2/conf.d/10-pdo.ini, /etc/php5/apache2/conf.d/20-curl.ini, /etc/php5/apache2/conf.d/20-gd.ini, /etc/php5/apache2/conf.d/20-json.ini, /etc/php5/apache2/conf.d/20-ldap.ini, /etc/php5/apache2/conf.d/20-mysql.ini, /etc/php5/apache2/conf.d/20-mysqli.ini, /etc/php5/apache2/conf.d/20-pdo_mysql.ini, /etc/php5/apache2/conf.d/20-readline.ini
PHP API	20121113
PHP Extension	20121212
Zend Extension	220121212
Zend Extension Build	API220121212,NTS
PHP Extension Build	API20121212,NTS
Debug Build	no
Thread Safety	disabled
Zend Signal Handling	disabled
Zend Memory Manager	enabled
Multibyte	provi

PHP Introduction

- PHP code is executed on the server, generating HTML that is sent to the client.
- The client will receive the result of running this script, but will not know what the underlying code is.

```
<html>
  <head><title>Example</title></head>
  <body>
     <?php
        echo "Hi.<br/><b>This is a PHP script!</b>";
     ?>
  </body>
</html>
```

```
<html>
   <head><title>Example</title></head>
   <body>
       Hi.<br/><b>This is a PHP script!</b>
   </body>
</html>
```



PHP Getting Started

- On Windows, you can download and install WAMP.
  - WampServer is a Windows web development environment. It allows you to create web applications with Apache2, PHP and a MySQL database. Alongside, PhpMyAdmin allows you to manage easily your database
  - Url: https://www.wampserver.com/en/

- On MacOS, you can download and install MAMP
  - MAMP is a free local server environment with Apache, Nginx, PHP, and MySQL.
  - Url: https://www.mamp.info/en/mac/

PHP `echo` and `print` statements

- With PHP, there are two basic ways to get output: `echo` and `print`.
- The differences are small:
  - `echo` has no return value;
  - `print` has a return value of 1 so it can be used in expressions;
  - `echo` can take multiple parameters (although such usage is rare);
  - `print` can take one argument;
  - `echo` is marginally faster than `print`.

```php
<?php
   echo "<h2>PHP is Fun!</h2>" ;
   print "Hello world!<br>";
   echo "I am about to learn PHP!";
?>
```

PHP Comments

- PHP supports Java and Unix shell-style comments.

```
<html>
   <head><title>Example</title></head>
   <body>
<?php
        echo 'This is a test'; // This is a one-line java style comment
           /* This is a multi line comment
              yet another line of comment */
           echo 'This is yet another test';
           echo 'One Final Test'; # This is a one-line shell-style comment
?>
   <h1>This is an <?php # echo 'simple';?> example</h1>
   <p>The header above will say 'This is an  example'.</p>
</body>
</html>
```

PHP Variable

- Variables are "containers" for storing information
- In PHP, a variable starts with the **$** sign, followed by the name of the variable

```
<?php
   $txt = "Hello world!";
   $x = 20;
?>
```

- In the example above, you can see that:
  - a variable does not need to be declared before adding a value to it.
  - you don't need to tell PHP what the data type of the variable is.
    - PHP automatically converts the variable to the correct data type, depending on its value.

PHP Variable

- A valid variable name starts with a letter (A-Z, a-z, or the bytes from 128 through 255) or underscore, followed by any number of letters, numbers, or underscores.
  - As a regular expression: [a-zA-Z_\x80-\xff][a-zA-Z0-9_\x80-\xff]*
    - Character code 128-255 are the extended ASCII codes

DEC	HEX	Symbol	Description
128	80	€	Euro sign
129	81		*Unused*
130	82	‚	Single low-9 quotation mark
131	83	ƒ	Latin small letter f with hook
132	84	„	Double low-9 quotation mark
133	85	…	Horizontal ellipsis
134	86	†	Dagger
135	87	‡	Double dagger
136	88	ˆ	Modifier letter circumflex accent
137	89	‰	Per mille sign
138	8A	Š	Latin capital letter S with caron
…			
255	FF	ÿ	Latin small letter y with diaeresis

PHP Variable

- Dynamic data types:

```php
<?php

   // Integers
   $a = 12345
   $b = -1221

   // Decimal
   $c = 1.234
   $d = 1.2e3;

   // String
   $e = "Hello world!";
   $f = 'Hello world!';

   // Boolean. Constants true or false (both are case-insensitive).
   $g = TRUE;

?>
```

Strings

- Strings in PHP are surrounded by either double quotation marks ("), or single quotation marks (').

```php
$e = "Hello world!";
$f = 'Hello world!';
```

- Double quoted strings perform action on special characters (e.g. when there is a variable in the string, it returns the value of the variable).

```php
$x = "John";
echo "Hello $x";
```

→ Hello John

- Single quoted strings does not perform such actions, it returns the string like it was written, with the variable name.

```php
$x = "John";
echo 'Hello $x';
```

→ Hello $x

Strings - Example

```php
<?php
   echo 'this is a simple string';

   echo 'You can also have embedded newlines in
       strings this way as it is
       okay to do';

   // Outputs: Arnold once said: "I'll be back"
   echo 'Arnold once said: "I\'ll be back"';

   // Outputs: You deleted C:\*.*?
   echo 'You deleted C:\\*.*?';

   // Outputs: This will not expand: \n a newline
   echo 'This will not expand: \n a newline';

   // Outputs: Variables do not $expand $either
   echo 'Variables do not $expand $either';

   ?>
```

Escape Character

Escape Character

String Operators

- Concatenation operator ('.') - which returns the concatenation of its right and left arguments.
- Concatenating assignment operator ('.=') - which appends the argument on the right side to the argument on the left side.

```php
<?php

   $a = "Hello ";
   $b = $a . "World!";   // now $b contains "Hello World!"

   $a .= "World!";       // now $a contains "Hello World!"

?>
```

Common String Functions

- strlen() - calculates the length of the string including all the whitespaces and special characters;
- str_word_count() - counts the number of words in a string;
- strrev() - reverses a string;
- strtoupper() - convert the text to uppercase;
- strtolower() - converts text to lower case;
- strcmp() - compares two strings in a case-sensitive manner;
- strchr() or strstr() - finds the first occurrence of a string in another string.

PHP Operators

- Operators are used to operate on values.
- There are four classifications of operators:
  - Arithmetic
  - Assignment
  - Comparison
  - Logical

Arithmetic Operators

Operator	Description	Example	Result
+	Addition	x = 2 y = x+2	y = 4
-	Subtraction	x = 2 y = 5-x	y = 3
*	Multiplication	x = 4 y = x*5	y = 20
/	Division	x = 15/5 y = 5/2	x = 3 y = 2.5
%	Modulus (division remainder)	x = 5%2 y = 10%5	x = 1 y = 0
++	Increment	x = 5 x++	x = 6
--	Decrement	x = 5 x--	x = 4

Assignment Operators

Operator	Example	Is the same as
=	x = y	x = y
+=	x += y	x = x+y
-=	x -= y	x = x-y
*=	x *= y	x = x*y
/=	x /= y	x = x/y
.=	x .= y	x = x.y
%=	x %= y	x = x%y

Comparison Operators

Operator	Description	Example
==	is equal to	5==8 returns false
!=	is not equal to	5!=8 returns true
>	is grater than	5>8 returns false
<	is less than	5<8 returns true
<=	is greater than or equal to	5>=8 returns false
>=	is less than or equal to	5<=8 returns true

Logical Operators

Operator	Description	Example
&&	and	x = 6 y = 3 (x<10 && y>1) returns true
||	or	x = 6 y = 3 (x==5 || y==1) returns false
!	not	x = 6 y = 3 !(x==y) returns true

PHP syntax

- The syntax of PHP is like Java.
  - Both languages use curly braces {} to define code blocks, uses semicolon to indicate the end of a statement and have similar control structures (like *if*, *else*, *while*, ...).
- In PHP, *keywords* (*if*, *else*, *while*, *echo*, ...),*classes*, *functions* and *user-defined functions* **are not** case-sensitive.
- However, all *variable names* **are** case-sensitive!

```
<?php
   FOR ($x = 0; $x <= 10; $x++) {
        If ($x == 3) cONTINUE;
        EcHo "The number is: $x <br>";
   }
?>
```

PHP Arrays

- An array in PHP is an ordered map.
  - A map is a type that associates values to keys.
- It can be created using the array() language construct.
  - It takes any number of comma-separated key => value pairs as arguments.

```php
<?php
    $mysites =  array(
       0 => "http://www.onlamp.com",
       1 => "http://www.oreilly.com"
    );

    // Using the short array syntax
    $site = "http://www.coggeshall.org";
    $mysites[2] = $site;
?>
```

Array Functions

Function	Description
asort	Sort an array and maintain index association
current	Return the current element in an array
each	Return the current key and value pair from an array and advance the array cursor
end	Set the internal pointer of an array to its last element
in_array	Checks if a value exists in an array
array_merge	Merge one or more arrays
array_multisort	Sort multiple or multi-dimensional arrays
array_pop	Pop the element off the end of array
array_push	Push one or more elements onto the end of array

- More functions: http://www.php.net/manual/en/ref.array.php

Example – One-Dimensional Array

```php
<?php
    $mysites =  array(
       "OnLamp" => "http://www.onlamp.com",
       "PHPNet" => "http://www.php.net",
       "Google" => "http://www.google.pt"
    );

    foreach ( $mysites as $value)  {
       echo  $value;
    }

    foreach ($mysites as $key => $value ) {
       echo  " $key\'s URL is $value ";
    }

    foreach($mysites as $key => $value ) {
       echo "<a href= $value > $key  </a > <br/>";
    }
?>
```

Example – Multidimensional Array

```php
<?php
   $Prods = array(
       array("Code" => "BOOK", "Description" => "Books", "Price" => 50),
       array("Code" => "DVDs", "Description" => "Movies", "Price" => 15),
       array("Code" => "CDs", "Description" => "Music", "Price" => 20)
    );

   for ($row = 0; $row < 3; $row++) {
?>
    <p>| <?=$Prods[$row]["Code"]?> | <?=$Prods[$row]["Description"]?> | <?=$Prods[$row]["Price"]?></p>
<?php
   }
?>
```

Example – Multidimensional Array

```php
<?php
   $Prods = array(
       array("Code" => "BOOK", "Description" => "Books", "Price" => 50),
       array("Code" => "DVDs", "Description" => "Movies", "Price" => 15),
       array("Code" => "CDs", "Description" => "Music", "Price" => 20)
    );

   for ($row = 0; $row < 3; $row++) {
       echo "<p> | $Prods[$row]['Code'] | $Prods[$row]['Description'] | $Prods[$row]['Price'] </p>"
   }
?>
```

User-defined Functions

- A function is defined using the **function** keyword, a name, a list of parameters (which might be empty) separated by commas (,) enclosed in parentheses, followed by the body of the function enclosed in curly braces.

```php
<?php
    function foo($arg_1, $arg_2, /* ..., */ $arg_n)
    {
        echo "Example function.\n";
        return $returnvalue;
    }
?>
```

- To call the function, simply write its name, placing the arguments in parentheses.

Example – Functions

```php
<?php

    function displayProductInfo($code,$desc,$price){
        echo "<p> | $code | $desc | $price </p>"
   }

   $Prods = array(
        array("Code" => "BOOK", "Description" => "Books", "Price" => 50),
        array("Code" => "DVDs", "Description" => "Movies", "Price" => 15),
        array("Code" => "CDs", "Description" => "Music", "Price" => 20)
    );

   for ($row = 0; $row < 3; $row++) {
        displayProductInfo($Prods[$row]['Code'] , $Prods[$row]['Description'], $Prods[$row]['Price']);
   }

?>
```

PHP Classes

- PHP includes a complete object model.
  - Some of its features are *visibility*, *abstract* and *final* classes and methods, *interfaces*, and *cloning*.
- PHP treats objects in the same way as references or handles, meaning that each variable contains an object reference rather than a copy of the entire object.

PHP Classes

- Basic class definitions begin with the keyword ***class***, followed by a class name, followed by a pair of curly braces which enclose the definitions of the properties and methods belonging to the class.
- The pseudo-variable ***$this*** is available when a method is called from within an object context.
  - ***$this*** is the value of the calling object.
- Within class methods:
  - non-static properties may be accessed by using **->** (Object Operator)

```
$this->id = $id;
```

  - Static properties are accessed by using the **::** (Double Colon)

```
ClassName::$staticProp;
```

PHP Classes - Constructor

- The ***__construct*** method is called on each new object created to initialize its properties.

```php
<?php
   class Fruit {
        public $name;
        public $color;

        function __construct($name) {
                  $this->name = $name;
        }
        function get_name() {
                  return $this->name;
        }
   }

   $apple = new Fruit("Apple");
   echo $apple->get_name();
?>
```